11. zkSNARK 생태계 연동 및 활용 방법 문서

개요

Circom 컴파일러가 생성하는 출력 파일들(.r1cs, .wasm, .sym)은 zkSNARK 증명 시스템의 핵심 구성 요소로, 다양한 증명 생성 라이브러리와 연동됩니다. 이 문서는 snarkjs와 rapidSnark를 중심으로 한 생태계 연동 과정을 상세히 분석하고, 실제 증명 시스템 구축을 위한 완전한 가이드를 제공합니다.

11.1 snarkjs와의 연동 과정 분석

snarkjs 아키텍처 개요

snarkjs는 JavaScript/TypeScript 환경에서 zkSNARK 증명을 생성하고 검증하는 완전한 라이브러리입니다. Circom 컴파일러와의 연동을 통해 다음과 같은 전체 워크플로우를 지원합니다:

```
.circom 파일 → Circom 컴파일 → .r1cs/.wasm/.sym → snarkjs → 증명/검증

↓ ↓ ↓ ↓ ↓

회로 정의 제약조건 생성 출력 파일 키 생성 증명 시스템
```

Circom 컴파일 결과물과 snarkjs 연동 매핑

1. R1CS 파일 (.r1cs) 활용

R1CS 파일은 snarkjs의 핵심 입력으로, 다음과 같은 방식으로 활용됩니다:

```
const snarkjs = require("snarkjs");

const fs = require("fs");
```

```
// R1CS 파일 정보 확인
async function analyzeR1CS() {
const r1csInfo = await snarkjs.r1cs.info("circuit.r1cs");
console.log("회로 정보:");
console.log(`- 제약 조건 수: ${r1csInfo.nConstraints}`);
console.log(`- 전체 신호 수: ${r1csInfo.nVars}`);
console.log(`- 공개 입력 수: ${r1csInfo.nPublicInputs}`);
console.log(`- 공개 출력 수: ${r1csInfo.nOutputs}`);
return r1csInfo;
}
```

2. WebAssembly 파일 (.wasm) 활용

WASM 파일은 증인(witness) 계산을 위해 사용됩니다:

```
const circomlib = require("circomlib");

// WASM 기반 증인 계산기 생성
async function createWitnessCalculator() {
const wasmBuffer = fs.readFileSync("circuit.wasm");
const witnessCalculator = await circomlib.WitnessCalculatorBuilder(wasmBuffe
return witnessCalculator;
}
```

```
// 입력으로부터 증인 계산
async function calculateWitness(input) {
const witnessCalculator = await createWitnessCalculator();
// 입력 검증
const inputSignals = witnessCalculator.getInputSignals();
console.log("필요한 입력 신호들:", inputSignals);
// 증인 계산 실행
const witness = await witnessCalculator.calculateWitness(input);
console.log(`계산된 증인 크기: ${witness.length}`);
return witness;
}
```

3. 심볼 파일 (.sym) 활용

심볼 파일은 디버깅과 신호 매핑에 활용됩니다:

```
// 심볼 파일 파싱
function parseSymbolFile() {
const symContent = fs.readFileSync("circuit.sym", "utf8");
const symbols = {};
symContent.split('\n').forEach(line => {
if (line.trim()) {
const [original, witness, nodeId, symbol] = line.split(',');
symbols[symbol] = {
```

```
original: parseInt(original),
witness: parseInt(witness),
nodeId: parseInt(nodeId)
};
}
});
return symbols;
}

// 증인 값과 신호 이름 매핑
function mapWitnessToSignals(witness, symbols) {
const signalValues = {};
Object.entries(symbols).forEach(([signalName, info]) => {
if (info.witness < witness.length) {
signalValues[signalName] = witness[info.witness].toString();
}
});
return signalValues;
}
```

키 생성 과정에서의 파일 활용 방법

1. Powers of Tau 세레모니

먼저 신뢰할 수 있는 설정을 위한 Powers of Tau 파일이 필요합니다:

```
// Powers of Tau 생성 (테스트용)
async function generatePowersOfTau() {
// 새로운 세레모니 시작
await snarkjs.powersOfTau.newAccumulator(
"bn128", // 곡선 타입
12, // 2^12 = 4096 제약조건 지원
"pot12_0000.ptau"
);
// 기여 추가 (실제 환경에서는 여러 참가자가 수행)
await snarkjs.powersOfTau.contribute(
"pot12_0000.ptau",
"pot12_0001.ptau",
"First contribution",
"random entropy"
);
// Phase 2 준비
await snarkjs.powersOfTau.preparePhase2(
"pot12_0001.ptau",
"pot12_final.ptau"
);
return "pot12_final.ptau";
}
```

2. 회로별 키 생성 (Setup)

R1CS 파일을 사용하여 회로별 증명/검증 키를 생성합니다:

```
// Groth16 키 생성
async function setupGroth16Keys() {
const ptauFile = "pot12_final.ptau";
const r1csFile = "circuit.r1cs";
// 초기 키 생성
await snarkjs.groth16.setup(r1csFile, ptauFile, "circuit_0000.zkey");
// Phase 2 기여 (실제 환경에서는 여러 참가자가 수행)
await snarkjs.zKey.contribute(
"circuit_0000.zkey",
"circuit_0001.zkey",
"First contribution",
"random entropy"
);
// 최종 키 생성
await snarkjs.zKey.beacon(
"circuit_0001.zkey",
"circuit_final.zkey",
"0102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f20",
10
);
```

```
// 검증 키 추출
const vKey = await snarkjs.zKey.exportVerificationKey("circuit_final.zkey");
fs.writeFileSync("verification_key.json", JSON.stringify(vKey, null, 2));
return {
zkeyFile: "circuit_final.zkey",
vkeyFile: "verification_key.json"
};
}
```

3. PLONK 키 생성 (대안)

```
// PLONK 키 생성 (Groth16의 대안)
async function setupPlonkKeys() {
const ptauFile = "pot12_final.ptau";
const r1csFile = "circuit.r1cs";
// PLONK 키 생성
await snarkjs.plonk.setup(r1csFile, ptauFile, "circuit.zkey");
// 검증 키 추출
const vKey = await snarkjs.zKey.exportVerificationKey("circuit.zkey");
fs.writeFileSync("verification_key.json", JSON.stringify(vKey, null, 2));
return {
zkeyFile: "circuit.zkey",
vkeyFile: "verification_key.json"
};
```

```
}
```

증명 생성 과정에서의 파일 활용 방법

1. 완전한 증명 생성 (fullProve)

가장 일반적인 증명 생성 방법입니다:

```
// 완전한 증명 생성 과정
async function generateProof(input) {
const wasmFile = "circuit.wasm";
const zkeyFile = "circuit_final.zkey";
try {
// 입력 검증
console.log("입력 데이터:", JSON.stringify(input, null, 2));
// 증명 생성 (증인 계산 + 증명 생성을 한 번에)
const { proof, publicSignals } = await snarkjs.groth16.fullProve(
input,
wasmFile,
zkeyFile
);
console.log("증명 생성 완료");
console.log("공개 신호:", publicSignals);
// 증명 데이터 저장
fs.writeFileSync("proof.json", JSON.stringify(proof, null, 2));
```

```
fs.writeFileSync("public.json", JSON.stringify(publicSignals, null, 2));
return { proof, publicSignals };
} catch (error) {
console.error("증명 생성 실패:", error.message);
// 디버깅을 위한 상세 정보 출력
await debugProofGeneration(input);
throw error;
}
}
```

2. 단계별 증명 생성

더 세밀한 제어가 필요한 경우:

```
// 단계별 증명 생성
async function generateProofStepByStep(input) {
// 1단계: 증인 계산
console.log("1단계: 증인 계산 중...");
const witnessCalculator = await createWitnessCalculator();
const witness = await witnessCalculator.calculateWitness(input);
// 증인 파일 저장
await snarkjs.wtns.calculate(input, "circuit.wasm", "witness.wtns");
// 2단계: 증명 생성
console.log("2단계: 증명 생성 중...");
```

```
const { proof, publicSignals } = await snarkjs.groth16.prove(
"circuit_final.zkey",
"witness.wtns"
);
// 3단계: 증명 검증 (선택적)
console.log("3단계: 증명 검증 중...");
const vKey = JSON.parse(fs.readFileSync("verification_key.json"));
const isValid = await snarkjs.groth16.verify(vKey, publicSignals, proof);
console.log("증명 유효성:", isValid);
return { proof, publicSignals, witness };
}
```

3. 배치 증명 생성

여러 증명을 효율적으로 생성:

```
// 배치 증명 생성
async function generateBatchProofs(inputs) {
const results = [];
const witnessCalculator = await createWitnessCalculator();
for (let i = 0; i < inputs.length; i++) {
console.log(`증명 ${i + 1}/${inputs.length} 생성 중...`);
try {
const { proof, publicSignals } = await snarkjs.groth16.fullProve(
inputs[i],
```

```
"circuit.wasm",
"circuit_final.zkey"
);
results.push({
index: i,
input: inputs[i],
proof,
publicSignals,
success: true
});
} catch (error) {
console.error(`증명 ${i + 1} 생성 실패:`, error.message);
results.push({
index: i,
input: inputs[i],
error: error.message,
success: false
});
}
}
return results;
}
```

검증 과정에서의 파일 활용 방법

1. 기본 검증

```
// 증명 검증
async function verifyProof() {
const vKey = JSON.parse(fs.readFileSync("verification_key.json"));
const proof = JSON.parse(fs.readFileSync("proof.json"));
const publicSignals = JSON.parse(fs.readFileSync("public.json"));
const isValid = await snarkjs.groth16.verify(vKey, publicSignals, proof);
console.log("증명 검증 결과:", isValid);
return isValid;
}
```

2. 스마트 컨트랙트 검증

블록체인에서 사용할 수 있는 Solidity 검증 컨트랙트 생성:

```
// Solidity 검증 컨트랙트 생성
async function generateSolidityVerifier() {
const vKey = JSON.parse(fs.readFileSync("verification_key.json"));
// Solidity 검증 컨트랙트 코드 생성
const solidityCode = await snarkjs.zKey.exportSolidityVerifier(
"circuit_final.zkey"
);
fs.writeFileSync("verifier.sol", solidityCode);
```

```
console.log("Solidity 검증 컨트랙트가 verifier.sol에 생성되었습니다.");
return  soliditycode;
}


// 스마트 컨트랙트용 증명 데이터 포맷팅
function  formatProofForSolidity(proof, publicSignals) {
return {
a: [proof.pi_a[0], proof.pi_a[1]],
b: [[proof.pi_b[0][1], proof.pi_b[0][0]], [proof.pi_b[1][1], proof.pi_b[1][0]]],
c: [proof.pi_c[0], proof.pi_c[1]],
inputs: publicSignals
};
}
```

JavaScript 환경에서의 zkSNARK 증명 시스템 구축 가이드

1. 완전한 프로젝트 구조

```
zksnark-project/
├── circuits/
| ├── circuit.circom # Circom 회로 정의
| ├── input.json # 테스트 입력
| └── compile.sh # 컴파일 스크립트
├── build/
```

```
| ├── circuit.r1cs # 컴파일된 제약조건
| ├── circuit.wasm # WebAssembly 코드
| ├── circuit.sym # 심볼 파일
| └── circuit_final.zkey # 증명 키
├── keys/
| ├── pot12_final.ptau # Powers of Tau
| └── verification_key.json # 검증 키
├── proofs/
| ├── proof.json # 생성된 증명
| └── public.json # 공개 신호
├── contracts/
| └── verifier.sol # Solidity 검증 컨트랙트
├── src/
| ├── setup.js # 키 생성 스크립트
| ├── prove.js # 증명 생성 스크립트
| ├── verify.js # 검증 스크립트
| └── utils.js # 유틸리티 함수
└── package.json
```

2. 패키지 설정 (package.json)

```
{
"name": "zksnark-project",
```

```
"version": "1.0.0",
"description": "zkSNARK proof system using Circom and snarkjs",
"main": "src/index.js",
"scripts": {
"compile": "cd circuits && ./compile.sh",
"setup": "node src/setup.js",
"prove": "node src/prove.js",
"verify": "node src/verify.js",
"test": "npm run compile && npm run setup && npm run prove && npm run verify"
},
"dependencies": {
"snarkjs": "^0.7.0",
"circomlib": "^2.0.5",
"ffjavascript": "^0.2.60"
},
"devDependencies": {
"mocha": "^10.0.0",
"chai": "^4.3.0"
}
}
```

3. 통합 워크플로우 클래스

```
// ZKProofSystem.js - 완전한 증명 시스템 클래스
```

```
class ZKProofSystem {
constructor(circuitName, options = {}) {
this.circuitName = circuitName;
this.options = {
buildDir: options.buildDir || './build',
keysDir: options.keysDir || './keys',
proofsDir: options.proofsDir || './proofs',
...options
};
this.files = {
r1cs: `${this.options.buildDir}/${circuitName}.r1cs`,
wasm: `${this.options.buildDir}/${circuitName}.wasm`,
sym: `${this.options.buildDir}/${circuitName}.sym`,
zkey: `${this.options.keysDir}/${circuitName}_final.zkey`,
vkey: `${this.options.keysDir}/verification_key.json`,
ptau: `${this.options.keysDir}/pot12_final.ptau`
};
}
// 전체 설정 과정
async setup() {
console.log("zkSNARK 시스템 설정 시작...");
// 1. Powers of Tau 확인/생성
if (!fs.existsSync(this.files.ptau)) {
```

```
console.log("Powers of Tau 생성 중...");
await this.generatePowersOfTau();
}
// 2. 회로 키 생성
console.log("회로 키 생성 중...");
await this.generateCircuitKeys();
// 3. 검증 키 추출
console.log("검증 키 추출 중...");
await this.extractVerificationKey();
console.log("설정 완료!");
}
// 증명 생성
async prove(input) {
console.log("증명 생성 시작...");
// 입력 검증
await this.validateInput(input);
// 증명 생성
const { proof, publicSignals } = await snarkjs.groth16.fullProve(
input,
this.files.wasm,
this.files.zkey
);
// 결과 저장
```

```
const timestamp = Date.now();
const proofFile = `${this.options.proofsDir}/proof_${timestamp}.json`;
const publicFile = `${this.options.proofsDir}/public_${timestamp}.json`;
fs.writeFileSync(proofFile, JSON.stringify(proof, null, 2));
fs.writeFileSync(publicFile, JSON.stringify(publicSignals, null, 2));
console.log("증명 생성 완료!");
return { proof, publicSignals, proofFile, publicFile };
}
// 증명 검증
async verify(proof, publicSignals) {
const vKey = JSON.parse(fs.readFileSync(this.files.vkey));
const isValid = await snarkjs.groth16.verify(vKey, publicSignals, proof);
console.log("증명 검증 결과:", isValid);
return isValid;
}
// 입력 검증
async validateInput(input) {
const witnessCalculator = await circomlib.WitnessCalculatorBuilder(
fs.readFileSync(this.files.wasm)
);
const inputSignals = witnessCalculator.getInputSignals();
// 필수 입력 확인
for (const signal of inputSignals) {
```

```
if (!(signal in input)) {
throw new Error(`필수 입력 신호 누락: ${signal}`);
}
}
// 추가 입력 확인
for (const signal in input) {
if (!inputSignals.includes(signal)) {
console.warn(`알 수 없는 입력 신호: ${signal}`);
}
}
}
// 디버깅 정보 출력
async debugInfo() {
console.log("=== zkSNARK 시스템 정보 ===");
// R1CS 정보
const r1csInfo = await snarkjs.r1cs.info(this.files.r1cs);
console.log("회로 정보:");
console.log(`- 제약 조건 수: ${r1csInfo.nConstraints}`);
console.log(`- 전체 신호 수: ${r1csInfo.nVars}`);
console.log(`- 공개 입력 수: ${r1csInfo.nPublicInputs}`);
console.log(`- 공개 출력 수: ${r1csInfo.nOutputs}`);
// 심볼 정보
if (fs.existsSync(this.files.sym)) {
```

```
const symbols = this.parseSymbolFile();
console.log(`- 심볼 수: ${Object.keys(symbols).length}`);
}
// 파일 크기 정보
console.log("파일 크기:");
Object.entries(this.files).forEach(([name, path]) => {
if (fs.existsSync(path)) {
const stats = fs.statSync(path);
console.log(`- ${name}: ${(stats.size / 1024 / 1024).toFixed(2)} MB`);
}
});
}
// 성능 벤치마크
async benchmark(input, iterations = 10) {
console.log(`성능 벤치마크 시작 (${iterations}회 반복)...`);
const times = {
prove: [],
verify: []
};
for (let i = 0; i < iterations; i++) {
// 증명 생성 시간 측정
const proveStart = Date.now();
const { proof, publicSignals } = await this.prove(input);
```

```
const proveTime = Date.now() - proveStart;
times.prove.push(proveTime);
// 검증 시간 측정
const verifyStart = Date.now();
await this.verify(proof, publicSignals);
const verifyTime = Date.now() - verifyStart;
times.verify.push(verifyTime);
console.log(`반복 ${i + 1}: 증명 ${proveTime}ms, 검증 ${verifyTime}ms`);
}
// 통계 계산
const avgProve = times.prove.reduce((a, b) => a + b) / times.prove.leng
const avgVerify = times.verify.reduce((a, b) => a + b) / times.verify.l
console.log("=== 벤치마크 결과 ===");
console.log(`평균 증명 시간: ${avgProve.toFixed(2)}ms`);
console.log(`평균 검증 시간: ${avgVerify.toFixed(2)}ms`);
return { avgProve, avgVerify, times };
}
}
```

4. 사용 예시

```
// 사용 예시
async function main() {
```

```
// 시스템 초기화
const zkSystem = new ZKProofSystem("multiplier");
// 설정 (최초 1회만)
await zkSystem.setup();
// 입력 데이터
const input = {
"a": "3",
"b": "4"
};
// 증명 생성
const { proof, publicSignals } = await zkSystem.prove(input);
// 증명 검증
const isValid = await zkSystem.verify(proof, publicSignals);
// 디버깅 정보
await zkSystem.debugInfo();
// 성능 벤치마크
await zkSystem.benchmark(input, 5);
}

main().catch(console.error);
```

이러한 snarkjs 연동 과정을 통해 JavaScript 환경에서 완전한 zkSNARK 증명 시스템을 구축할 수 있으며, 웹 애플리케이션, Node.js 서버, 그리고 블록체인 dApp에서 활용할 수 있습니다.

11.2 ra

pidSnark 및 C++ 환경 연동 분석

rapidSnark 아키텍처 개요

rapidSnark는 C++로 구현된 고성능 zkSNARK 증명 생성 라이브러리로, Circom 컴파일러가 생성하는 C++ 코드와 직접 연동되어 네이티브 성능의 증명 생성을 제공합니다. JavaScript 기반 snarkjs 대비 10-100배 빠른 증명 생성 속도를 달성할 수 있습니다.

```
Circom 컴파일 → C++ 코드 → rapidSnark 연동 → 고성능 증명 생성

↓ ↓ ↓ ↓

.circom .cpp/.hpp 네이티브 컴파일 최적화된 증명
```

C++ 기반 고성능 증명 생성을 위한 rapidSnark 연동 방법

1. rapidSnark 설치 및 환경 설정

```
# rapidSnark 소스 코드 클론
git clone https://github.com/iden3/rapidsnark.git
cd rapidsnark

# 의존성 설치 (Ubuntu/Debian)
sudo apt-get update
sudo apt-get install build-essential cmake libgmp-dev libsodium-dev nasm

# 빌드 설정
```

```
git submodule init
git submodule update
mkdir build && cd build

# CMake 설정 (최적화 옵션 포함)
cmake .. -DCMAKE_BUILD_TYPE=Release -DUSE_PROFILING=OFF -DUSE_ASM=ON

# 컴파일
make -j$(nproc)
```

2. Circom 컴파일러와 rapidSnark 연동 구조

Circom 컴파일러가 생성하는 C++ 코드는 rapidSnark와 다음과 같은 구조로 연동됩니다:

생성된 C++ 파일 구조

```
circuit_cpp/
├── circuit.cpp # 메인 회로 구현
├── circuit.dat # 회로 데이터 (바이너리)
├── circuit.hpp # 헤더 파일
├── main.cpp # 실행 진입점
├── circom.hpp # Circom 런타임 헤더
├── calcwit.cpp # 증인 계산 구현
├── calcwit.hpp # 증인 계산 헤더
├── fr.cpp # 필드 연산 구현 (필드별)
├── fr.hpp # 필드 연산 헤더
```

```
└── Makefile # 빌드 스크립트
```

핵심 데이터 구조 연동

```
// circom.hpp - Circom 런타임 핵심 구조
class Circom_CalcWit {
public:
// 필드 연산 타입 정의
typedef  struct {
uint64_t  v[4]; // BN128의 경우 256비트
} FrElement;
// 컴포넌트 구조
typedef  struct {
uint32_t offset;
uint32_t size;
uint32_t subcomponents;
uint32_t subcomponents_offset;
uint32_t signal_offset;
uint32_t signal_offset_jump;
uint32_t component_name;
uint32_t component_father;
uint32_t idFather;
uint32_t subcomponent_offset;
```

```
} Circom_Component;

// 메인 데이터 멤버

FrElement *signalValues; // 신호 값 배열

Circom_Component *componentMemory; // 컴포넌트 메모리

uint32_t *circuitConstants; // 회로 상수

std::map<u32,IOFieldDefPair> templateInsId2IOSignalInfo; // I/O 매핑

// 핵심 메서드

void calculateWitness(std::map<std::string, std::vector<BigInt>>& input);

void getWitness(uint32_t index, PBigInt value);

void setSignal(uint32_t cIdx, uint32_t sIdx, PBigInt value);

};
```

3. rapidSnark 프로버 연동 구현

```
// prover_integration.cpp - rapidSnark 프로버 연동

#include "rapidsnark/prover.h"

#include "circom.hpp"

#include "calcwit.hpp"


class CircomRapidSnarkProver {

private:

std::unique_ptr<Circom_CalcWit> calcWit;

std::string zkeyPath;

std::string datPath;
```

```
public:
CircomRapidSnarkProver(const std::string&  datFile, const std::string&  zkeyFile
: datPath(datFile), zkeyPath(zkeyFile) {
// Circom 증인 계산기 초기화
calcWit = std::make_unique<Circom_CalcWit>(datFile);
}
// 증명 생성 메인 함수
ProofResult generateProof(const InputMap&  inputs) {
try {
// 1단계: 증인 계산
auto witness = calculateWitness(inputs);
// 2단계: rapidSnark를 통한 증명 생성
auto proof = generateProofFromWitness(witness);
return ProofResult{true, proof, ""};
} catch (const std::exception& e) {
return ProofResult{false, {}, e.what()};
}
}
private:
// 증인 계산
std::vector<BigInt> calculateWitness(const InputMap&  inputs) {
// 입력 설정
std::map<std::string, std::vector<BigInt>> circomInputs;
```

```
for (const  auto& [key, value] : inputs) {
circomInputs[key] = {BigInt(value)};
}
// Circom 증인 계산 실행
calcWit->calculateWitness(circomInputs);
// 증인 추출
std::vector<BigInt> witness;
uint32_t witnessSize = calcWit->getWitnessSize();
for (uint32_t i = 0; i < witnessSize; i++) {
BigInt value;
calcWit->getWitness(i, &value);
witness.push_back(value);
}
return witness;
}
// rapidSnark를 통한 증명 생성
Proof generateProofFromWitness(const std::vector<BigInt>&  witness) {
// rapidSnark 프로버 초기화
auto prover = rapidsnark::Prover::create(zkeyPath);
// 증인을 rapidSnark 형식으로 변환
auto rapidWitness = convertWitnessFormat(witness);
// 증명 생성
auto proof = prover->prove(rapidWitness);
```

```
return proof;
}
// 증인 형식 변환
rapidsnark::Witness convertWitnessFormat(const std::vector<BigInt>&  witness) {
rapidsnark::Witness rapidWitness;
rapidWitness.resize(witness.size());
for (size_t i = 0; i < witness.size(); i++) {
// BigInt를 rapidSnark의 필드 원소로 변환
rapidWitness[i] = rapidsnark::Fr(witness[i].toString());
}
return rapidWitness;
}
};
```

4. 필드별 최적화 연동

rapidSnark는 다양한 필드에 대해 최적화된 구현을 제공합니다:

BN128 필드 최적화

```
// fr_bn128.cpp - BN128 필드 특화 구현
namespace bn128 {
// Montgomery 형태 연산
void  Fr_add(PFrElement r, PFrElement a, PFrElement b) {
// 인라인 어셈블리를 통한 최적화
```

```
asm volatile (
"movq %1, %%rax\n\t"
"addq %2, %%rax\n\t"
"movq %%rax, %0\n\t"
: "=m" (r->v[0])
: "m" (a->v[0]), "m" (b->v[0])
: "rax"
);
// ... 추가 구현
}
void Fr_mul(PFrElement r, PFrElement a, PFrElement b) {
// Montgomery 곱셈 최적화
montgomery_mul(r->v, a->v, b->v);
}
}
```

Goldilocks 필드 최적화

```
// fr_goldilocks.cpp - Goldilocks 필드 특화 구현
namespace goldilocks {
// 64비트 네이티브 연산 활용
void Fr_add(uint64_t* r, const uint64_t* a, const uint64_t* b) {
*r = (*a + *b) % GOLDILOCKS_PRIME;
}
```

```
void Fr_mul(uint64_t* r, const uint64_t* a, const uint64_t* b) {
__uint128_t result = (__uint128_t)(*a) * (*b);
*r = result % GOLDILOCKS_PRIME;
}
}
```

생성된 C++ 코드의 컴파일 및 최적화 옵션 활용 가이드

1. 기본 컴파일 설정

```
# Makefile - 최적화된 컴파일 설정
CXX = g++
CXXFLAGS = -std=c++17 -O3 -march=native -mtune=native
CXXFLAGS += -DUSE_ASM -DNDEBUG -fPIC -pthread

# 필드별 최적화 플래그
ifeq ($(FIELD), bn128)
CXXFLAGS += -DUSE_BN128 -DUSE_MONTGOMERY
endif

ifeq ($(FIELD), goldilocks)
CXXFLAGS += -DUSE_GOLDILOCKS -DUSE_NATIVE_64BIT
endif
```

```
# 링크 라이브러리
LDFLAGS = -lgmp -lsodium -lcrypto -lpthread

# OpenMP 병렬 처리 (선택적)
ifdef USE_OPENMP
CXXFLAGS += -fopenmp
LDFLAGS += -lgomp
endif

# 소스 파일
SOURCES = circuit.cpp calcwit.cpp fr.cpp main.cpp
OBJECTS = $(SOURCES:.cpp=.o)
TARGET = circuit_prover

# 빌드 규칙
$(TARGET): $(OBJECTS)
$(CXX)  $(OBJECTS) -o $@  $(LDFLAGS)

%.o: %.cpp
$(CXX)  $(CXXFLAGS) -c $< -o $@

clean:
rm -f $(OBJECTS)  $(TARGET)
```

```
.PHONY: clean
```

2. CMake 기반 빌드 시스템

```
# CMakeLists.txt - 고급 빌드 설정
cmake_minimum_required(VERSION 3.16)
project(CircomRapidSnark)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 최적화 옵션
if(CMAKE_BUILD_TYPE STREQUAL "Release")
set(CMAKE_CXX_FLAGS_RELEASE "-O3 -march=native -mtune=native -DNDEBUG")
endif()

# 필드 타입 설정
set(FIELD_TYPE "bn128" CACHE STRING "Field type (bn128, goldilocks, bls12381)")

if(FIELD_TYPE STREQUAL "bn128")
add_definitions(-DUSE_BN128 -DUSE_MONTGOMERY)
elseif(FIELD_TYPE STREQUAL "goldilocks")
```

```
add_definitions(-DUSE_GOLDILOCKS -DUSE_NATIVE_64BIT)
endif()

# 어셈블리 최적화
option(USE_ASM "Use assembly optimizations" ON)
if(USE_ASM)
add_definitions(-DUSE_ASM)
endif()

# OpenMP 병렬 처리
option(USE_OPENMP "Use OpenMP for parallelization" OFF)
if(USE_OPENMP)
find_package(OpenMP REQUIRED)
target_link_libraries(circuit_prover OpenMP::OpenMP_CXX)
endif()

# 의존성 라이브러리
find_package(PkgConfig REQUIRED)
pkg_check_modules(GMP REQUIRED gmp)
pkg_check_modules(SODIUM REQUIRED libsodium)

# 실행 파일 생성
add_executable(circuit_prover
```

```
circuit.cpp

calcwit.cpp

fr.cpp

main.cpp

)


target_link_libraries(circuit_prover

${GMP_LIBRARIES}

${SODIUM_LIBRARIES}

crypto

pthread

)


# rapidSnark 연동

option(USE_RAPIDSNARK "Link with rapidSnark library" ON)

if(USE_RAPIDSNARK)

find_library(RAPIDSNARK_LIB rapidsnark PATHS /usr/local/lib)

if(RAPIDSNARK_LIB)

target_link_libraries(circuit_prover ${RAPIDSNARK_LIB})

add_definitions(-DUSE_RAPIDSNARK)

endif()

endif()
```

3. 프로파일링 기반 최적화 (PGO)

```bash
#!/bin/bash
# pgo_build.sh - Profile-Guided Optimization 빌드

# 1단계: 프로파일링 정보 수집을 위한 빌드
g++ -std=c++17 -O3 -march=native -fprofile-generate \
circuit.cpp calcwit.cpp fr.cpp main.cpp \
-lgmp -lsodium -lcrypto -pthread \
-o circuit_prover_profile

# 2단계: 대표적인 입력으로 프로파일링 실행
echo "프로파일링 데이터 수집 중..."
for i in {1..100}; do
./circuit_prover_profile input_sample_$i.json > /dev/null
done

# 3단계: 프로파일링 정보를 활용한 최적화 빌드
g++ -std=c++17 -O3 -march=native -fprofile-use \
circuit.cpp calcwit.cpp fr.cpp main.cpp \
-lgmp -lsodium -lcrypto -pthread \
-o circuit_prover_optimized
```

```
echo "PGO 최적화 완료!"
```

4. 링크 시간 최적화 (LTO)

```
# lto_build.sh - Link Time Optimization
g++ -std=c++17 -O3 -march=native -flto -fuse-linker-plugin \
circuit.cpp calcwit.cpp fr.cpp main.cpp \
-lgmp -lsodium -lcrypto -pthread \
-o circuit_prover_lto
```

대규모 회로에서의 성능 최적화 및 메모리 관리 기법

1. 메모리 풀링 시스템

```
// memory_pool.hpp - 효율적인 메모리 관리
class CircomMemoryPool {
private:
struct MemoryBlock {
void* ptr;
size_t size;
bool inUse;
};
std::vector<MemoryBlock> blocks;
std::mutex poolMutex;
public:
```

```
// 메모리 블록 사전 할당
void  preallocate(size_t  totalSize, size_t  blockSize) {
size_t numBlocks = totalSize / blockSize;
blocks.reserve(numBlocks);
for (size_t i = 0; i < numBlocks; i++) {
void* ptr = aligned_alloc(64, blockSize); // 64바이트 정렬
if (ptr) {
blocks.push_back({ptr, blockSize, false});
}
}
}
// 메모리 할당
void*  allocate(size_t  size) {
std::lock_guard<std::mutex> lock(poolMutex);
for (auto& block : blocks) {
if (!block.inUse && block.size >= size) {
block.inUse = true;
return  block.ptr;
}
}
// 풀에서 사용 가능한 블록이 없으면 새로 할당
return  aligned_alloc(64, size);
}
```

```
// 메모리 해제
void deallocate(void* ptr) {
std::lock_guard<std::mutex> lock(poolMutex);
for (auto& block : blocks) {
if (block.ptr == ptr) {
block.inUse = false;
return;
}
}
// 풀 외부 메모리는 직접 해제
free(ptr);
}
};

// 전역 메모리 풀 인스턴스
extern CircomMemoryPool g_memoryPool;
```

2. 스트리밍 증인 계산

```
// streaming_witness.hpp - 대용량 회로를 위한 스트리밍 처리
class StreamingWitnessCalculator {
private:
static constexpr size_t CHUNK_SIZE = 1024 * 1024; // 1MB 청크
```

```
std::unique_ptr<Circom_CalcWit> calcWit;

std::vector<FrElement> witnessBuffer;

size_t currentChunk;

size_t totalWitnessSize;

public:

StreamingWitnessCalculator(const std::string&  datFile)

: calcWit(std::make_unique<Circom_CalcWit>(datFile)),

currentChunk(0) {

totalWitnessSize = calcWit->getWitnessSize();

witnessBuffer.reserve(CHUNK_SIZE);

}

// 청크 단위로 증인 계산

bool  calculateNextChunk() {

if (currentChunk * CHUNK_SIZE >= totalWitnessSize) {

return false; // 모든 청크 처리 완료

}

size_t startIdx = currentChunk * CHUNK_SIZE;

size_t endIdx = std::min(startIdx + CHUNK_SIZE, totalWitnessSize);

witnessBuffer.clear();

witnessBuffer.reserve(endIdx - startIdx);

// 해당 청크의 증인 값들 계산

for (size_t i = startIdx; i < endIdx; i++) {

FrElement value;
```

```
calcWit->getWitness(i, &value);
witnessBuffer.push_back(value);
}
currentChunk++;
return true;
}
// 현재 청크의 증인 데이터 반환
const std::vector<FrElement>&  getCurrentChunk() const {
return witnessBuffer;
}
// 진행률 반환
double  getProgress() const {
return static_cast<double>(currentChunk * CHUNK_SIZE) / totalWitnessSize;
}
};
```

3. 병렬 처리 최적화

```
// parallel_prover.hpp - 병렬 증명 생성
class ParallelCircomProver {
private:
std::vector<std::unique_ptr<Circom_CalcWit>> workers;
ThreadPool threadPool;
public:
```

```
ParallelCircomProver(const std::string&  datFile, size_t  numThreads = 0) {
if (numThreads == 0) {
numThreads = std::thread::hardware_concurrency();
}
// 워커 스레드별로 독립적인 계산기 생성
workers.reserve(numThreads);
for (size_t i = 0; i < numThreads; i++) {
workers.push_back(std::make_unique<Circom_CalcWit>(datFile));
}
threadPool.resize(numThreads);
}
// 배치 증명 생성
std::vector<ProofResult> generateBatchProofs(
const std::vector<InputMap>&  inputs) {
std::vector<std::future<ProofResult>> futures;
futures.reserve(inputs.size());
// 각 입력에 대해 비동기 증명 생성 작업 스케줄링
for (size_t i = 0; i < inputs.size(); i++) {
auto future = threadPool.enqueue([this, i, &inputs]() {
size_t workerIdx = i % workers.size();
return  generateSingleProof(inputs[i], workerIdx);
});
futures.push_back(std::move(future));
```

```
}
// 모든 작업 완료 대기
std::vector<ProofResult> results;
results.reserve(inputs.size());
for (auto& future : futures) {
results.push_back(future.get());
}
return results;
}
private:
ProofResult generateSingleProof(const InputMap&  input, size_t  workerIdx) {
try {
auto& calcWit = workers[workerIdx];
// 증인 계산
std::map<std::string, std::vector<BigInt>> circomInputs;
for (const  auto& [key, value] : input) {
circomInputs[key] = {BigInt(value)};
}
calcWit->calculateWitness(circomInputs);
// 증인 추출 및 증명 생성
auto witness = extractWitness(*calcWit);
auto proof = generateProofFromWitness(witness);
return ProofResult{true, proof, ""};
```

```
} catch (const std::exception& e) {
return ProofResult{false, {}, e.what()};
}
}
};
```

4. 캐시 최적화

```
// cache_optimization.hpp - 캐시 친화적 데이터 구조
class CacheOptimizedCircuit {
private:
// 캐시 라인 크기에 맞춘 데이터 정렬
alignas(64) struct SignalBlock {
FrElement signals[8]; // 64바이트 = 8 * 8바이트
};
std::vector<SignalBlock> signalBlocks;
std::vector<uint32_t> accessPattern; // 메모리 접근 패턴 최적화
public:
// 신호 접근 패턴 분석 및 최적화
void  optimizeMemoryLayout() {
// 접근 빈도 분석
std::map<uint32_t, uint32_t> accessFrequency;
analyzeAccessPattern(accessFrequency);
```

```
// 빈도 기반 메모리 재배치
reorderSignalsByFrequency(accessFrequency);
// 캐시 라인 정렬
alignSignalBlocks();
}
// 프리페치를 통한 성능 향상
void prefetchSignals(uint32_t startIdx, uint32_t count) {
for (uint32_t i = 0; i < count; i += 8) { // 캐시 라인 단위
uint32_t blockIdx = (startIdx + i) / 8;
if (blockIdx < signalBlocks.size()) {
__builtin_prefetch(&signalBlocks[blockIdx], 0, 3);
}
}
}
};
```

5. 성능 모니터링 및 프로파일링

```
// performance_monitor.hpp - 성능 모니터링
class PerformanceMonitor {
private:
struct Metrics {
std::chrono::high_resolution_clock::time_point startTime;
std::chrono::duration<double> witnessTime;
```

```
std::chrono::duration<double> proofTime;

size_t memoryUsage;

size_t peakMemoryUsage;

};

Metrics currentMetrics;

std::vector<Metrics> history;

public:

void  startMeasurement() {

currentMetrics.startTime = std::chrono::high_resolution_clock::now();

currentMetrics.memoryUsage = getCurrentMemoryUsage();

}

void  recordWitnessCalculation() {

auto now = std::chrono::high_resolution_clock::now();

currentMetrics.witnessTime = now - currentMetrics.startTime;

updateMemoryUsage();

}

void  recordProofGeneration() {

auto now = std::chrono::high_resolution_clock::now();

currentMetrics.proofTime = now - currentMetrics.startTime - currentMetrics.witne

updateMemoryUsage();

history.push_back(currentMetrics);

}

// 성능 통계 출력
```

```
void  printStatistics() const {
if (history.empty()) return;
double avgWitnessTime = 0, avgProofTime = 0;
size_t avgMemoryUsage = 0, maxMemoryUsage = 0;
for (const  auto& metrics : history) {
avgWitnessTime += metrics.witnessTime.count();
avgProofTime += metrics.proofTime.count();
avgMemoryUsage += metrics.memoryUsage;
maxMemoryUsage = std::max(maxMemoryUsage, metrics.peakMemoryUsage);
}
size_t count = history.size();
avgWitnessTime /= count;
avgProofTime /= count;
avgMemoryUsage /= count;
std::cout << "=== 성능 통계 (" << count << "회 측정) ===" << std::endl;
std::cout << "평균 증인 계산 시간: " << avgWitnessTime << "초" << std::endl;
std::cout << "평균 증명 생성 시간: " << avgProofTime << "초" << std::endl;
std::cout << "평균 메모리 사용량: " << (avgMemoryUsage / 1024 / 1024) << "MB" << s
std::cout << "최대 메모리 사용량: " << (maxMemoryUsage / 1024 / 1024) << "MB" << s
}
private:
size_t  getCurrentMemoryUsage() const {
// /proc/self/status에서 메모리 사용량 읽기 (Linux)
```

```
std::ifstream file("/proc/self/status");

std::string line;

while (std::getline(file, line)) {

if (line.substr(0, 6) == "VmRSS:") {

std::istringstream iss(line);

std::string label, value, unit;

iss >> label >> value >> unit;

return std::stoull(value) * 1024; // KB to bytes

}

}

return  0;

}

void  updateMemoryUsage() {

size_t current = getCurrentMemoryUsage();

currentMetrics.memoryUsage = current;

currentMetrics.peakMemoryUsage = std::max(currentMetrics.peakMemoryUsage, curren

}

};
```

6. 실제 사용 예시

```
// main.cpp - 최적화된 증명 생성 시스템 사용 예시

int  main(int  argc, char*  argv[]) {
```

```
try {
// 성능 모니터 초기화
PerformanceMonitor monitor;
// 메모리 풀 사전 할당 (1GB)
g_memoryPool.preallocate(1024 * 1024 * 1024, 1024 * 1024);
// 병렬 프로버 초기화
ParallelCircomProver prover("circuit.dat", 8); // 8개 스레드
// 입력 데이터 로드
std::vector<InputMap> inputs = loadInputsFromFile("batch_inputs.json");
// 배치 증명 생성
monitor.startMeasurement();
auto results = prover.generateBatchProofs(inputs);
monitor.recordProofGeneration();
// 결과 저장
saveResultsToFile(results, "batch_proofs.json");
// 성능 통계 출력
monitor.printStatistics();
std::cout << "배치 증명 생성 완료: " << results.size() << "개" << std::endl;
} catch (const std::exception& e) {
std::cerr << "오류: " << e.what() << std::endl;
return 1;
}
return 0;
```

```
}
```

이러한 rapidSnark 연동과 C++ 최적화 기법을 통해 대규모 zkSNARK 회로에서도 실용적인 성능의 증명 생성이 가능하며, 프로덕션 환경에서 요구되는 처리량과 응답 시간을 달성할 수 있습니다.